



Assumption University of Thailand

Vincent Mary School of Engineering, Science and Technology

CSX3007 – Computer Architecture

Semester 2/2025

Computer Architecture Term Project Report

Submitted to

Asst. Prof. Dr. Anilkumar K Gopalakrishnan

Submitted by

Kaung Khant Lin (ID: 6540131)

Submission Date: March 7, 2026

Table of Contents

1. Introduction.....	1
PART I – NON-PIPELINED EXECUTION MODEL.....	2
2. Instruction Set Architecture (ISA) Design.....	2
2.1 Architectural Assumptions.....	2
2.2 Final 32-Bit Instruction Format.....	2
2.3 Opcode Mapping.....	3
3. The Assembler (Instruction Decoder).....	4
3.1 Architectural Enforcement and Validation.....	4
3.2 Two's Complement Encoding and Instruction Packing.....	4
4. The Execution Engine (ALU Simulation).....	5
4.1 Hardware Sign-Extension.....	5
4.2 ALU Operations.....	5
4.3 Double-Width Arithmetic (HI/LO Abstraction).....	6
5. Simulation Trace and Performance Evaluation (CPI).....	7
PART II – PIPELINED EXECUTION MODEL.....	8
6. Pipelined Architecture Abstraction.....	8
7. Pipeline Execution Timeline.....	8
8. Hazard Detection and Forwarding Resolution.....	9
9. Learning Outcomes and Limitations.....	10
APPENDIX A – CPU SIMULATOR SOURCE CODE.....	11

Instruction Set Architecture (ISA) Simulator

1. Introduction

This term project implements a strict Instruction Set Architecture (ISA) simulator for a 32-bit Central Processing Unit (CPU). Developed in Java, the simulator models a basic processor datapath that operates exclusively on 32-bit signed two's complement integers. The primary objective is to transition theoretical computer architecture concepts into a functional software model, demonstrating how hardware constraints such as rigid bit-field boundaries, architectural execution rules, and gate-level propagation delays dictate the structure of machine language and processor performance.

The modeled ISA features eight 32-bit General-Purpose Registers (GPRs, `r0`–`r7`) and supports foundational arithmetic operations (`mov`, `add`, `sub`, `mul`, `div`). The design rigorously adheres to 32-bit hardware limitations. Specifically, it addresses double-width 64-bit outputs during multiplication by utilizing `r7` as a dedicated high-order/remainder register, mirroring the dedicated HI/LO register behavior found in the MIPS architecture.

To evaluate processor throughput, the project is divided into two developmental phases:

- **Part I – Non-Pipelined Execution Model:** A sequential execution simulator that models how instructions are encoded, decoded, and executed one at a time, establishing a baseline Cycles Per Instruction (CPI) measurement based on gate-delay latencies.
- **Part II – Pipelined Execution Model:** An architectural extension implementing a multi-stage pipeline to demonstrate instruction throughput scaling and data hazard resolution.

PART I – NON-PIPELINED EXECUTION MODEL

2. Instruction Set Architecture (ISA) Design

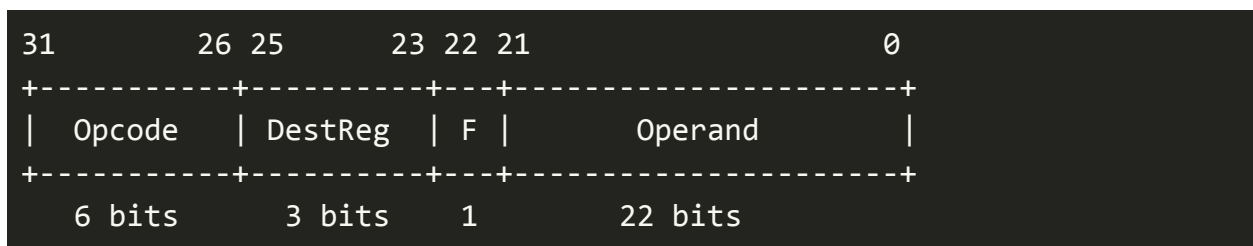
2.1 Architectural Assumptions

The processor follows standard 32-bit architecture assumptions:

- All registers and datapaths are exactly 32 bits wide.
- Arithmetic follows two's complement signed behavior.
- Register `r7` is strictly reserved to store the upper 32 bits of a multiplication product and the remainder of a division operation. It cannot be used as the explicit destination register for `mul` or `div`.

2.2 Final 32-Bit Instruction Format

To maximize the size of embedded immediate constants without exceeding the 32-bit hardware limit, the instruction word is rigidly partitioned into four fields.



Field	Size	Bits	Purpose
Opcode	6 bits		Specifies the ALU operation.
Dest Reg	3 bits		Selects the target register (r0-r7).

Flag	1 bit	[22]	0 = Register operand, 1 = Immediate operand.
Operand	22 bits	[21:0]	Signed immediate constant or source register address.

2.3 Opcode Mapping

The Control Unit decodes instructions using the following assignments:

Instruction	Opcode (Binary)	Opcode (Decimal)
mov	000001	1
add	000010	2
sub	000011	3
mul	000100	4
div	000101	5
end	111111	63

3. The Assembler (Instruction Decoder)

The internal assembler translates human-readable assembly instructions into 32-bit machine code.

3.1 Architectural Enforcement and Validation

To prevent illegal machine states, the assembler enforces ISA constraints before execution begins. If a program attempts to violate the r7 accumulator rule, the assembler rejects it:

```
if ((opcode == OP_MUL || opcode == OP_DIV) && destReg == 7) {
    throw new IllegalArgumentException("r7 cannot be destination for
    MUL/DIV.");
}
```

3.2 Two's Complement Encoding and Instruction Packing

Java represents negative integers natively using 32-bit two's complement. If a negative immediate value (e.g., -1) is bitwise-packed directly, its leading 1s will overwrite the opcode and register fields. To ensure strict structural isolation, the assembler applies a logical AND mask (0x3FFFFFF) to preserve only the lower 22 bits before shifting and packing the final word:

```
int encodedInstruction = ((opcode & 0x3F) << OP_CODE_SHIFT)
    | ((destReg & 0x7) << REG_SHIFT)
    | ((flag & 0x1) << FLAG_SHIFT)
    | (operandField & IMMEDIATE_MASK);
```

4. The Execution Engine (ALU Simulation)

The execution engine simulates a non-pipelined CPU datapath. Instructions execute sequentially (Fetch → Decode → Execute → Write Back), meaning no overlap occurs.

4.1 Hardware Sign-Extension

When an immediate operand is extracted, Java treats it as a positive 32-bit integer because the upper 10 bits are initialized to 0. To perform correct two's complement arithmetic, the simulator replicates hardware sign-extension via arithmetic shifting:

```
value2 = (operandField << 10) >> 10;
```

The left shift removes residual structural bits, and the arithmetic right shift (>>) naturally propagates the sign bit across the upper 10 bits, sending the correct signed value to the ALU.

4.2 ALU Operations

The datapath dynamically alters register states based on the decoded opcode. Basic arithmetic routing is handled synchronously:

```
switch (opcode) {  
    case OP_ADD:  
        registers[destReg] += value2;  
        break;  
    case OP_SUB:  
        registers[destReg] -= value2;  
        break;  
}
```

4.3 Double-Width Arithmetic (HI/LO Abstraction)

Multiplying two 32-bit integers produces a 64-bit result, which risks silent truncation in standard software. The simulator forces a strict double-width execution pathway by casting to `long`, then routing the upper 32 bits to `r7` (HI) and the lower 32 bits to the destination register (LO).

```
long product = (long) registers[destReg] * (long) value2;
registers[7] = (int) (product >> 32);
registers[destReg] = (int) product;
```

5. Simulation Trace and Performance Evaluation (CPI)

Processor performance is quantified using Cycles Per Instruction (CPI). Because this is a non-pipelined sequential model, total execution time is the sum of individual instruction latencies derived from a theoretical gate-delay model:

Instruction	Cycles	Adder/Multiplier/Divider Type
mov, add, sub	1	Carry-propagate adder
mul	4	Multi-cycle multiplier array
div	8	N-bit ripple divider

The example instruction sequence used in the evaluation is adapted from a classroom demonstration program used to illustrate arithmetic datapath behavior.

Simulation Execution Trace:

```

PC[0] ->  mov r1, 543 : 000001 001 1 0000000000001000011111 1
PC[1] ->  mov r2, 675 : 000001 010 1 0000000000001010100011 1
PC[2] ->  sub r1, r2  : 000011 001 0 0000000000000000000010 1
PC[3] ->  mov r3, r1  : 000001 011 0 0000000000000000000001 1
PC[4] ->  mul r3, 654 : 000100 011 1 0000000000001010001110 4
PC[5] ->  mov r4, r3  : 000001 100 0 0000000000000000000011 1
PC[6] ->  mov r5, r7  : 000001 101 0 0000000000000000000011 1
PC[7] ->  div r4, 13  : 000101 100 1 0000000000000000000011 8

```

The 8-instruction program consumed exactly 18 clock cycles.

Steady-State Performance:

$$CPI = \frac{\text{Total Cycles}}{\text{Total Instructions}} = \frac{18}{8} = 2.25$$

PART II – PIPELINED EXECUTION MODEL

6. Pipelined Architecture Abstraction

To demonstrate throughput improvement, the CPU transitions from a sequential model to an overlapping pipelined execution model.

Architectural Modification: The physical multi-cycle structural delays established in Part I (e.g., `mul` costing 4 cycles) are intentionally abstracted away. The pipeline assumes a uniform 4-stage model (Instruction Fetch, Decode, Execute, Write Back), where each stage resolves in exactly one clock cycle. This creates a pipeline that does not experience structural stalls.

7. Pipeline Execution Timeline

Pipelining increases throughput by overlapping execution. The total execution time is modeled by the formula: $TotalCycles = N + k - 1$, where N is the number of instructions and k is the pipeline depth.

Simulation Execution Trace:

	1	2	3	4	5	6	7	8	9	10	11
1 <code>mov r1, 543</code>	:IF	ID	EX	WB							
2 <code>mov r2, 675</code>	:	IF	ID	EX	WB						
3 <code>sub r1, r2</code>	:		IF	ID	EX	WB					
4 <code>mov r3, r1</code>	:			IF	ID	EX	WB				
5 <code>mul r3, 654</code>	:				IF	ID	EX	WB			
6 <code>mov r4, r3</code>	:					IF	ID	EX	WB		
7 <code>mov r5, r7</code>	:						IF	ID	EX	WB	
8 <code>div r4, 13</code>	:							IF	ID	EX	WB

The trace confirms that the 8-instruction sequence ($N = 8$, $k = 4$), which required 18 cycles non-pipelined, completes in $8 + 4 - 1 = 11$ clock cycles when pipelined.

8. Hazard Detection and Forwarding Resolution

Simultaneous execution introduces Read-After-Write (RAW) Data Hazards. A RAW hazard occurs when an instruction attempts to read an operand from a register before the preceding instruction has reached the WB stage to update it.

In a two-operand ISA, arithmetic instructions (like `mul r3 654`) inherently read the destination register (`r3`) before overwriting it. The simulator tracks these intersections to flag hazards. To prevent pipeline stalls, the simulator assumes the presence of a Forwarding Unit, which physically routes the output of the EX stage back to the ALU input for the next cycle.

```
if (registersWritten.contains(readReg)) {
    System.out.printf(
        "%d in instructions %d and %d caused RAW hazard and is
solved by forwarding.\n",
        readReg, i, i + 1
    );
}
```

Hazard Detection Output:

```
RAW hazard details:
r2 in instructions 2 and 3 caused RAW hazard and is solved by
forwarding.
r1 in instructions 3 and 4 caused RAW hazard and is solved by
forwarding.
r3 in instructions 4 and 5 caused RAW hazard and is solved by
forwarding.
r3 in instructions 5 and 6 caused RAW hazard and is solved by
forwarding.
```

This output provides empirical proof that the simulator correctly maps data dependencies down to the register read/write level, successfully resolving Distance-1 hazards via forwarding.

9. Learning Outcomes and Limitations

This project successfully modeled both the physical constraints of instruction encoding and the performance scaling of pipelined architecture.

Limitations: The simulator focuses exclusively on integer arithmetic. Memory hierarchy operations (load/store), Branch logic (Control Hazards), and Structural Hazards within the pipeline are abstracted out to focus purely on core ALU datapath mechanisms and RAW data forwarding.

Learning Outcomes:

- **Bit-Level Discipline:** Implemented structural field isolation within a 32-bit boundary without data bleed.
- **Hardware Emulation:** Modeled hardware-accurate sign-extension and 64-bit mathematical overflow handling.
- **Performance Analysis:** Derived and contrasted CPI metrics by calculating physical gate-delays versus theoretical pipeline throughput.

APPENDIX A – CPU SIMULATOR SOURCE CODE

The complete Java implementation of the ISA simulator is provided in this appendix. The code implements the assembler, execution engine, CPI analysis, and pipelined execution model described in the report. The simulator accepts assembly instructions from the console, encodes them into a 32-bit instruction format, executes them sequentially, and then simulates pipelined execution with RAW hazard detection and forwarding.

CpuSimulator.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class CpuSimulator {

    //
    -----
    ---
    // Architectural State
    //
    -----
    ---

    private int[] registers = new int[8];
    private int totalCycles = 0;
    private int instructionCount = 0;

    // Chronological execution log
    private List<String> executionLog = new ArrayList<>();

    // Instruction Opcodes (6-bit)
    private static final int OP_MOV = 1;
    private static final int OP_ADD = 2;
    private static final int OP_SUB = 3;
    private static final int OP_MUL = 4;
    private static final int OP_DIV = 5;
    private static final int OP_END = 63;
```

```

// Bit-shifting constants
private static final int OPCODE_SHIFT = 26;
private static final int REG_SHIFT = 23;
private static final int FLAG_SHIFT = 22;
private static final int IMMEDIATE_MASK = 0x3FFFFFF;

//
-----
---
// Startup Header
//
-----
---
    public void printStartupHeader() {

System.out.println("=====
===");
        System.out.println("Details of the Instruction Set
Architecture (ISA)");

System.out.println("=====
===");
        System.out.println("This is a 32-bit ISA and supports 32-bit
arithmetic.");
        System.out.println("There are eight 32-bit General Purpose
Registers (r0 to r7).");
        System.out.println("Register r7 stores upper-half
multiplication results and division remainders.");
        System.out.println("Hence r7 cannot be used as destination in
MUL or DIV.");

System.out.println("-----
---");
        System.out.println("Syntax: <opcode> <dest_reg>
<source_reg/immediate>");
        System.out.println("Example: mov r1 3");
        System.out.println("Type 'end 0 0' to terminate.");
    }

```

```

System.out.println("=====
===\n");
    }

    //
-----
---
    // Assembler
    //
-----
---
    public int assemble(String instruction) {

        String[] parts =
instruction.trim().toLowerCase().split("\\s+");

        String mnemonic = parts[0];
        if (mnemonic.equals("end")) {
            return (OP_END << OPCODE_SHIFT);
        }

        int opcode = switch (mnemonic) {
            case "mov" -> OP_MOV;
            case "add" -> OP_ADD;
            case "sub" -> OP_SUB;
            case "mul" -> OP_MUL;
            case "div" -> OP_DIV;
            default -> throw new IllegalArgumentException("Unknown
mnemonic: " + mnemonic);
        };

        if (parts.length < 2 || !parts[1].startsWith("r")) {
            throw new IllegalArgumentException("Missing or malformed
destination register.");
        }

        int destReg = Integer.parseInt(parts[1].substring(1));

        if ((opcode == OP_MUL || opcode == OP_DIV) && destReg == 7) {

```

```

        throw new IllegalArgumentException("r7 cannot be
destination for MUL/DIV.");
    }

    if (parts.length < 3) {
        throw new IllegalArgumentException("Missing second
operand.");
    }

    int flag;
    int operandField;

    String op2 = parts[2];
    if (op2.startsWith("r")) {
        flag = 0;
        operandField = Integer.parseInt(op2.substring(1)) & 0x7;
    } else {
        flag = 1;
        int imm = Integer.parseInt(op2);
        operandField = imm & IMMEDIATE_MASK;
    }

    return ((opcode & 0x3F) << OPCODE_SHIFT)
        | ((destReg & 0x7) << REG_SHIFT)
        | ((flag & 0x1) << FLAG_SHIFT)
        | (operandField & IMMEDIATE_MASK);
}

//
-----
---
// Execution Engine
//
-----
---
private int getCycleCost(int opcode) {
    return switch (opcode) {
        case OP_MOV, OP_ADD, OP_SUB -> 1;
        case OP_MUL -> 4;
    };
}

```

```

        case OP_DIV -> 8;
        default -> 0;
    };
}

public int execute(int instruction) {

    int opcode = (instruction >>> OPCODE_SHIFT) & 0x3F;
    if (opcode == OP_END) return -1;

    int destReg = (instruction >>> REG_SHIFT) & 0x7;
    int flag = (instruction >>> FLAG_SHIFT) & 0x1;
    int operandField = instruction & IMMEDIATE_MASK;

    int value2 = (flag == 1)
        ? (operandField << 10) >> 10
        : registers[operandField & 0x7];

    switch (opcode) {
        case OP_MOV:
            registers[destReg] = value2;
            executionLog.add(formatReg(destReg));
            break;

        case OP_ADD:
            registers[destReg] += value2;
            executionLog.add(formatReg(destReg));
            break;

        case OP_SUB:
            registers[destReg] -= value2;
            executionLog.add(formatReg(destReg));
            break;

        case OP_MUL:
            long product = (long) registers[destReg] * (long)
value2;
            registers[7] = (int) (product >> 32);
            registers[destReg] = (int) product;
    }
}

```

```

        executionLog.add(
            String.format("r7: r%d = %d [%s]", destReg,
product, to32BitBinary((int)product))
        );
        break;

    case OP_DIV:
        if (value2 != 0) {
            int quotient = registers[destReg] / value2;
            int remainder = registers[destReg] % value2;
            registers[destReg] = quotient;
            registers[7] = remainder;

            executionLog.add(
                String.format("r%d = %d [%s] r7: %d
[%s]",
                                destReg, registers[destReg],
to32BitBinary(registers[destReg]),
                                registers[7],
to32BitBinary(registers[7])
                )
            );
        } else {
            executionLog.add("Hardware Exception: Division by
zero.");
        }
        break;
    }

    int cycles = getCycleCost(opcode);
    totalCycles += cycles;
    instructionCount++;
    return cycles;
}

//
-----
---
```

```

// Part II: Pipelined Execution & Hazard Detection
//
-----
---
    public void simulatePipeline(List<Integer> memoryList,
List<String> programList, Scanner scanner) {
        System.out.println("\nPipelined Execution of the Program");
        System.out.println("It is assumed that the CPU has 4-pipeline
stages: IF (Instruction Fetch) ID (Instruction decoding), EX
(Execution) and WB (Write back). And each stage completes within one
clock cycle and RAW hazard (any) will be solved with FORWARDING (from
the o/p register of ALU to the i/p of next instruction's ALU stage)
without causing any stall.\n");

        int n = 0;
        for (int instr : memoryList) {
            if (((instr >>> OPCODE_SHIFT) & 0x3F) == OP_END) break;
            n++;
        }

        if (n == 0) return;

        System.out.print("                ");
        for (int i = 1; i <= n + 3; i++) {
            System.out.printf("%-5d", i);
        }
        System.out.println();

        for (int i = 0; i < n; i++) {
            String decoded = formatDecoded(programList.get(i));
            String prefix = String.format("%d %s :", (i + 1),
decoded);
            System.out.printf("%-15s", prefix);

            for (int space = 0; space < i; space++) {
                System.out.print("        ");
            }
            System.out.println("IF | ID | EX | WB");
        }
    }

```

```

    int pipelineCycles = n + 4 - 1;
    System.out.println("\nPipelined execution took " +
pipelineCycles + " clock cycles for the program execution.");

    System.out.println("RAW hazard details:");
    boolean hazardFound = false;

    for (int i = 1; i < n; i++) {
        int currentInstr = memoryList.get(i);
        int currentOpcode = (currentInstr >>> OPCODE_SHIFT) &
0x3F;

        int currentDest = (currentInstr >>> REG_SHIFT) & 0x7;
        int currentFlag = (currentInstr >>> FLAG_SHIFT) & 0x1;
        int currentSrc = currentInstr & 0x7;

        int prevInstr = memoryList.get(i - 1);
        int prevOpcode = (prevInstr >>> OPCODE_SHIFT) & 0x3F;
        int prevDest = (prevInstr >>> REG_SHIFT) & 0x7;

        List<Integer> registersWritten = new ArrayList<>();
        registersWritten.add(prevDest);
        if (prevOpcode == OP_MUL || prevOpcode == OP_DIV) {
            registersWritten.add(7);
        }

        List<Integer> registersRead = new ArrayList<>();
        if (currentOpcode != OP_MOV) {
            registersRead.add(currentDest);
        }
        if (currentFlag == 0) {
            registersRead.add(currentSrc);
        }

        for (int readReg : registersRead) {
            if (registersWritten.contains(readReg)) {
                System.out.printf("r%d in instructions %d and %d
caused RAW hazard and is solved by forwarding.\n",
readReg, i, i + 1);
            }
        }
    }
}

```

```

        hazardFound = true;
    }
}

if (!hazardFound) {
    System.out.println("No RAW hazards detected.");
}

System.out.print("\nPress 'y' to continue or 'n' to stop the
simulation : ");
String response = scanner.nextLine().trim();
if (response.equalsIgnoreCase("n")) {
    System.exit(0);
}

//
-----
---
// Formatting Helpers
//
-----
---

private String formatReg(int r) {
    return String.format("r%d = %d [%s]", r, registers[r],
to32BitBinary(registers[r]));
}

private String to32BitBinary(int value) {
    return String.format("%32s",
Integer.toBinaryString(value)).replace(' ', '0');
}

public String formatInstructionBinary(int instruction) {
    String bin = to32BitBinary(instruction);
    return bin.substring(0, 6) + " "
        + bin.substring(6, 9) + " "
        + bin.substring(9, 10) + " "

```

```

        + bin.substring(10);
    }

    private String formatDecoded(String decoded) {
        String[] parts = decoded.split("\\s+");
        if (parts.length == 3) {
            return parts[0] + " " + parts[1] + ", " + parts[2];
        }
        return decoded;
    }

    public void printExecutionLog() {
        System.out.println("\nValues of registers after the execution
of the instruction set:");
        System.out.println("During the execution of the above code
sequence, the values of registers would be varied in the following
way:");
        for (String line : executionLog) {
            System.out.println(line);
        }
        System.out.println();
    }

    public void printCPI() {
        if (instructionCount > 0) {
            double cpi = (double) totalCycles / instructionCount;
            System.out.println("CPI of the program.....");
            System.out.printf("CPI = %.2f\n", cpi);
        } else {
            System.out.println("CPI = 0.00");
        }
    }

    //
    -----
    ---
    // Main
    //
    -----

```

```

---
public static void main(String[] args) {

    CpuSimulator cpu = new CpuSimulator();
    cpu.printStartupHeader();

    Scanner scanner = new Scanner(System.in);
    List<String> programList = new ArrayList<>();
    List<Integer> memoryList = new ArrayList<>();

    System.out.println("Enter instructions line by line:");
    int lineCount = 1;

    while (true) {
        System.out.print(lineCount + " ");
        String input = scanner.nextLine().trim();
        if (input.isEmpty()) continue;

        try {
            int encoded = cpu.assemble(input);
            programList.add(input);
            memoryList.add(encoded);
            lineCount++;

            if (input.toLowerCase().startsWith("end")) break;
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
            System.out.println("Please try again.");
        }
    }

    System.out.println("\nProgram Assembly Complete. Commencing
Execution...\n");
    System.out.printf("%-10s %-18s %-36s %s\n", "PC", "Decoded:",
"Encoded instructions (32-bit):", "Clock cycles");

    System.out.println("-----
-----");

```

```

int pc = 0;
while (pc < memoryList.size()) {
    int instruction = memoryList.get(pc);
    String decoded = programList.get(pc);

    int cycles = cpu.execute(instruction);
    if (cycles == -1) break;

    System.out.printf(
        "PC[%d] ->  %-15s :  %-36s %d\n",
        pc,
        cpu.formatDecoded(decoded),
        cpu.formatInstructionBinary(instruction),
        cycles
    );

    pc++;
}

cpu.printExecutionLog();
cpu.printCPI();

cpu.simulatePipeline(memoryList, programList, scanner);
scanner.close();
}
}

```